Question 1

A software company is developing a `PaymentSystem` that requires all payment providers to implement a common interface with methods such as `makePayment(amount)`, `checkPaymentStatus(transactionId)`, and `cancelPayment(transactionId)`.

However, a third-party legacy payment service called `QuickPay` provides a different API with methods such as `process(amount)`, `getStatus(transactionCode)`, and `voidTransaction(transactionCode)`.

a) Identify the design pattern being used. (2 marks)

The design pattern being used is the **Adapter Pattern**.

b) Provide code for the main classes (8 marks)

Java Implementation

```java
// Common concrete class used by PaymentSystem
public class PaymentProvider {
    public void makePayment(double amount) {
        System.out.println("Default payment provider making payment...");
    }

    public String checkPaymentStatus(String transactionId) {
        return "Default status for transaction: " + transactionId;
    }

    public void cancelPayment(String transactionId) {
        System.out.println("Default payment provider cancelling payment...");
    }
}

// Third-party legacy service
public class QuickPay {
    public void process(double amount) {
        System.out.println("Processing amount via QuickPay: " + amount);
    }

    public String getStatus(String transactionCode) {
        return "Status for QuickPay transaction: " + transactionCode;
    }

    public void voidTransaction(String transactionCode) {
        System.out.println("Voiding QuickPay transaction: " + transactionCode);
```

```
    }
}

// Adapter extends the concrete class to integrate QuickPay
public class QuickPayAdapter extends PaymentProvider {
    private QuickPay quickPay;

    public QuickPayAdapter(QuickPay quickPay) {
        this.quickPay = quickPay;
    }

    @Override
    public void makePayment(double amount) {
        quickPay.process(amount);
    }

    @Override
    public String checkPaymentStatus(String transactionId) {
        return quickPay.getStatus(transactionId);
    }

    @Override
    public void cancelPayment(String transactionId) {
        quickPay.voidTransaction(transactionId);
    }
}

// Client class
public class Client {
    public static void main(String[] args) {
        QuickPay quickPay = new QuickPay();
        PaymentProvider paymentProvider = new
    QuickPayAdapter(quickPay);

        paymentProvider.makePayment(100.0);
        System.out.println(paymentProvider.
    checkPaymentStatus("TXN123"));
        paymentProvider.cancelPayment("TXN123");
    }
}
```

Python Implementation

```
# Common concrete class used by PaymentSystem
class PaymentProvider:
    def make_payment(self, amount: float):
        print("Default payment provider making payment...")

```

```
    def check_payment_status(self, transaction_id: str) -> str:
        return f"Default status for transaction: {transaction_id}"

    def cancel_payment(self, transaction_id: str):
        print("Default payment provider cancelling payment ...")

# Third-party legacy service
class QuickPay:
    def process(self, amount: float):
        print(f"Processing amount via QuickPay: {amount}")

    def get_status(self, transaction_code: str) -> str:
        return f"Status for QuickPay transaction: {transaction_code}"

    def void_transaction(self, transaction_code: str):
        print(f"Voiding QuickPay transaction: {transaction_code}")

# Adapter inherits from the concrete class to integrate QuickPay
class QuickPayAdapter(PaymentProvider):
    def __init__(self, quick_pay: QuickPay):
        self.quick_pay = quick_pay

    def make_payment(self, amount: float):
        self.quick_pay.process(amount)

    def check_payment_status(self, transaction_id: str) -> str:
        return self.quick_pay.get_status(transaction_id)

    def cancel_payment(self, transaction_id: str):
        self.quick_pay.void_transaction(transaction_id)

# Client code
if __name__ == "__main__":
    quick_pay = QuickPay()
    payment_provider = QuickPayAdapter(quick_pay)

    payment_provider.make_payment(100.0)
    print(payment_provider.check_payment_status("TXN123"))
    payment_provider.cancel_payment("TXN123")
```

Question 2: Short Questions (5 marks)

a) (2 marks) What is the main intent of the Observer pattern?
The main intent of the Observer pattern is to define a one-to-many dependency between objects so that when one object (the subject) changes its state, all its dependents (observers) are notified and updated automatically.

b) (2 marks) In the Observer Pattern, what is the role of the Subject?
The role of the Subject is to maintain a list of its dependent observers and provide an interface to attach (register) and detach (unregister) them. It is responsible for sending notifications to all registered observers whenever its internal state changes.

c) (1 mark) Which pattern (Observer, Adapter, or Singleton) is most suitable if you want to allow multiple instances to respond to the same event?
The **Observer** pattern.